

OMNICART MARKETPLACE

Complete Features, Services & System Scope

Frontend + Spring Boot Backend + MySQL Database

Project domain: General E-commerce Marketplace

Prepared as the consolidated implementation scope for the OmniCart DBMS project.

1. Project Scope

OmniCart Marketplace is a general e-commerce marketplace covering customer management, seller management, hierarchical categories, product cataloging, inventory, AI-based visual search, dynamic pricing, order management, payment-method information, shipping status, reporting, and database-level integrity.

The submitted project requirements explicitly define customer and address management, product/category management, product embeddings for image-based similarity search, an append-only price ledger, order management, and transaction-time price locking. The supplied ERD maps these requirements to Customer, Address, Seller, Category, Product, Product Embeddings, Price Ledger, Order, and Order Details entities.

Technology Stack

Layer	Technology	Purpose
Frontend	HTML + CSS + JavaScript	Marketplace UI and REST API consumption
Backend	Java + Spring Boot	REST APIs, business logic, security, transactions
Persistence	Spring Data JPA / Hibernate	Object-relational mapping
Database	MySQL 8+	Relational storage, constraints, views, procedures, functions, triggers
Build	Maven	Dependency and build management
API	REST / JSON	Frontend-backend communication

2. System Architecture

```

Frontend
|
| REST / JSON
v
Spring Boot Controllers
|
v
Service Layer
|
v
Repository / JPA Layer
|
v
MySQL Database
  
```

Recommended Backend Package Structure

```

com.omnicart
├── controller
├── service
├── service.impl
├── repository
├── entity
├── dto
├── request
├── response
├── mapper
├── exception
├── config
├── security
└── util
  
```

3. Customer Features & Services

Frontend Features

- Customer registration and login entry point
- Customer profile and profile editing
- Email and phone management
- Multiple saved delivery addresses
- Address-type selection
- Customer dashboard
- Order history and order details
- Order status tracking
- Checkout address selection

Backend Services

CustomerService
 CustomerProfileService
 CustomerContactService
 AddressService
 AddressValidationService
 CustomerOrderService
 CustomerDashboardService

Database Objects

Customer
 CustomerEmail
 CustomerPhone
 Address
 AddressType

Customer APIs

POST /api/customers
 GET /api/customers
 GET /api/customers/{id}
 PUT /api/customers/{id}
 POST /api/customers/{id}/emails
 DELETE /api/customers/{id}/emails/{emailId}
 POST /api/customers/{id}/phones
 DELETE /api/customers/{id}/phones/{phoneId}
 POST /api/customers/{id}/addresses
 GET /api/customers/{id}/addresses
 PUT /api/addresses/{id}
 DELETE /api/addresses/{id}
 GET /api/customers/{id}/orders
 GET /api/customers/{id}/dashboard

4. Seller Features & Services

Frontend Features

- Seller registration and profile
- Company information and GSTIN
- Seller dashboard
- Product creation and editing
- Product images

- Inventory view and stock updates
- Low-stock and out-of-stock views
- Price management and price history
- Seller order list
- Seller sales information

Backend Services

SellerService
SellerProfileService
SellerProductService
SellerOrderService
SellerAnalyticsService
SellerDashboardService

Database Objects

Seller
Product
ProductImage
PriceLedger

Seller APIs

POST /api/sellers
GET /api/sellers/{id}
PUT /api/sellers/{id}
GET /api/sellers/{sellerId}/products
GET /api/sellers/{sellerId}/inventory
GET /api/sellers/{sellerId}/orders
GET /api/sellers/{sellerId}/sales
GET /api/sellers/{sellerId}/dashboard

5. Category Management

- Create, view, update, and deactivate categories
- Parent-child category hierarchy
- Category tree navigation
- Subcategory retrieval
- Products by category
- Category-level product counts

Backend Services

CategoryService
CategoryHierarchyService
CategoryProductService

Database

Category(CategoryID, CategoryName, CategoryDescription, ParentCategoryID, Level)

APIs

POST /api/categories
GET /api/categories
GET /api/categories/{id}
PUT /api/categories/{id}
DELETE /api/categories/{id}
GET /api/categories/tree

GET /api/categories/{id}/children

GET /api/categories/{id}/products

6. Product Catalog & Product Management

- Product listing and product detail pages
- Create/update/deactivate product
- Product name and description
- Seller association
- Category association
- Stock availability
- Current price display
- Related/similar products
- Product images
- Product embeddings

Backend Services

ProductService

ProductCatalogService

ProductManagementService

ProductImageService

InventoryService

Database

Product

ProductImage

Seller

Category

APIs

POST /api/products

GET /api/products

GET /api/products/{id}

PUT /api/products/{id}

DELETE /api/products/{id}

GET /api/products/category/{categoryId}

GET /api/products/seller/{sellerId}

GET /api/products/low-stock

7. Product Image Management

- Upload multiple product images
- View product images
- Delete image
- Set primary image
- Change image display order
- Store image path/URL rather than large binary data in the relational database

ProductImageService

ImageStorageService

POST /api/products/{productId}/images

GET /api/products/{productId}/images

PUT /api/product-images/{imageId}

DELETE /api/product-images/{imageId}

8. Product Search

- Keyword search
- Search by product name
- Search by description
- Category filter
- Seller filter
- Price range filter
- Availability filter
- Pagination

ProductSearchService
CatalogSearchService

GET /api/products/search?keyword=
GET /api/products/search?keyword=&category=
GET /api/products/search?keyword=&seller=
GET /api/products/search?keyword=&minPrice=&maxPrice=

9. AI Visual Search & Product Embeddings

The visual-search subsystem stores AI-generated feature vectors in a dedicated product-embedding structure and supports image-based similarity search. Under the supplied ERD, Product and ProductEmbeddings are intended to have a 1:1 relationship.

Frontend Features

- Image upload
- Image preview
- Visual search action
- Similar-product results
- Similarity/result ranking
- Availability and category filtering of results

Backend Services

EmbeddingService
ProductEmbeddingService
VisualSearchService
SimilaritySearchService

Database

ProductEmbeddings(EmbeddingID, ProductID, FeatureVector)

APIs

POST /api/products/{productId}/embedding
GET /api/products/{productId}/embedding
PUT /api/products/{productId}/embedding
DELETE /api/products/{productId}/embedding
POST /api/search/visual

Flow

Image -> Frontend -> Spring Boot -> Embedding Generation -> Feature Vector
-> ProductEmbeddings -> Similarity Search -> Similar Products

10. Inventory Management

- View current stock
- Increase stock
- Decrease stock
- Low-stock detection
- Out-of-stock detection
- Stock validation during checkout
- Transactional stock reduction
- Stock restoration for valid cancellation

InventoryService

StockManagementService

GET /api/products/{id}/stock

PUT /api/products/{id}/stock

POST /api/products/{id}/stock/add

POST /api/products/{id}/stock/remove

GET /api/products/low-stock

11. Dynamic Pricing & Price Ledger

The price subsystem keeps historical pricing separate from the main Product record. New prices are recorded as new ledger entries; historical entries are not overwritten.

Features

- Set a new price
- Read current price
- Read complete price history
- Record timestamped price changes
- Store price source
- Track price fluctuations
- Use the latest applicable ledger record as the current price

Backend Services

PriceService

PriceLedgerService

CurrentPriceService

PriceHistoryService

OrderPricingService

Database

PriceLedger(LedgerID, ProductID, Timestamp, Price, PriceFluctuation, Source)

APIs

POST /api/products/{id}/prices

GET /api/products/{id}/prices

GET /api/products/{id}/current-price

GET /api/products/{id}/price-history

Append-only rule

Old price -> remains unchanged

New price -> INSERT new PriceLedger row

Historical rows -> no UPDATE / no DELETE

12. Cart Module (Application-Level Extension)

A persistent cart is not explicitly represented in the submitted ERD. It may be implemented as an application-level extension without changing the documented core entities.

- Add product to cart
- Remove product
- Change quantity
- View cart
- Calculate subtotal
- Proceed to checkout

CartService

CartItemService

Typical frontend pages: Cart, CartItem, CartSummary

13. Checkout Module

- Review selected products
- Select delivery address
- Confirm quantities
- Display current transaction price
- Select payment method
- Review total
- Place order

CheckoutService

OrderService

InventoryService

PriceService

PaymentService

14. Order Management

Frontend Features

- Place order
- Order confirmation
- Order history
- Order details
- Order tracking
- Order cancellation where allowed
- Display exact purchase price per item
- Display calculated order total

Backend Services

OrderService

OrderDetailsService

OrderCalculationService

OrderStatusService

OrderHistoryService

ShippingService

Database Objects

Orders

OrderDetails

OrderPayment

APIs

```
POST /api/orders
GET /api/orders/{id}
GET /api/customers/{customerId}/orders
PUT /api/orders/{id}/shipping-status
POST /api/orders/{id}/cancel
```

15. Order Placement Transaction & Price Locking

This is a critical business transaction. The order, line items, exact purchase prices, stock change, and payment-method record should be coordinated as one database transaction.

```
BEGIN TRANSACTION
1. Validate customer
2. Validate delivery address and ownership
3. Validate products and quantities
4. Lock relevant inventory rows
5. Retrieve latest valid PriceLedger records
6. Verify sufficient stock
7. Create Orders row
8. Create OrderDetails rows
9. Copy ExactLedgerPrice for every line item
10. Reduce stock
11. Create OrderPayment record
12. Calculate order total
13. COMMIT
```

Any failure -> ROLLBACK

Concurrency Rule

When two customers try to buy the last unit simultaneously, only one order should be able to consume that unit. Use transactional row locking such as SELECT ... FOR UPDATE or JPA pessimistic locking.

16. Order Details & Total Calculation

OrderDetails resolves the Order-Product many-to-many relationship. The key line-item values are Quantity and ExactLedgerPrice. The supplied ERD identifies TotalAmount as a derived attribute.

```
OrderDetails:
OrderID
ProductID
Quantity
ExactLedgerPrice
LedgerID (optional traceability)

Line subtotal = Quantity * ExactLedgerPrice
Order Total = SUM(Line subtotals)
```

17. Payment Module

- Select payment method
- Attach payment method to order
- Read payment method for an order
- Keep sensitive payment credentials out of the project database

PaymentService

OrderPaymentService

OrderPayment(OrderID, PaymentMethod)

18. Shipping Status & Order Cancellation

Recommended status flow:

PLACED -> CONFIRMED -> SHIPPED -> DELIVERED

Possible cancellation paths:

PLACED -> CANCELLED

CONFIRMED -> CANCELLED

Services

OrderStatusService

ShippingService

Cancellation transaction

Verify order -> verify allowed state -> restore stock when applicable

-> update order state -> preserve OrderDetails historical price -> COMMIT

19. Customer Dashboard

Customer Dashboard

|— Profile

|— Contact Information

|— Saved Addresses

|— Recent Orders

|— Active Orders

|— Order History

|— Total Orders

|— Total Spending

CustomerDashboardService

GET /api/customers/{id}/dashboard

20. Seller Dashboard

Seller Dashboard

|— Seller Profile

|— Total Products

|— Inventory

|— Low Stock

|— Out of Stock

|— Orders

|— Sales

|— Current Prices

|— Price History

|— Product Management

SellerDashboardService

SellerAnalyticsService

GET /api/sellers/{id}/dashboard

21. Marketplace / Admin Features (Application Extension)

The submitted requirements define marketplace database operations but do not prescribe an administrator UI. The following are implementation extensions around the documented domain.

- Customer management
- Seller management
- Category management
- Product management
- Order monitoring
- Inventory monitoring
- Price monitoring
- Marketplace statistics
- Reporting and analytics

AdminService

MarketplaceService

ReportingService

AnalyticsService

22. MySQL Database Scope

Core Tables

Area	Core tables
Customer	Customer, CustomerEmail, CustomerPhone, Address, AddressType
Marketplace	Seller, Category
Catalog	Product, ProductImage, ProductEmbeddings
Pricing	PriceLedger
Orders	Orders, OrderDetails, OrderPayment

Core Relationships

Customer 1:N Address

Customer 1:N Orders

Seller 1:N Product

Category 1:N Product

Category 1:N Category

Product 1:N ProductImage

Product 1:1 ProductEmbeddings

Product 1:N PriceLedger

Orders 1:N OrderDetails

Product 1:N OrderDetails

Orders 1:1 / 1:N OrderPayment

23. Database Constraints & Integrity

- Primary keys on all entity identifiers
- Foreign keys for all documented relationships
- UNIQUE for fields such as customer email and seller GSTIN where applicable
- NOT NULL for required business attributes
- CHECK constraints for non-negative stock and prices and positive order quantities
- ProductEmbeddings.ProductID unique to enforce the intended 1:1 relationship
- Careful ON DELETE / ON UPDATE behavior so historical orders and price records are not accidentally removed

24. Database Indexing

Entity	Recommended indexes	Reason
Customer	Email, phone	Lookup and uniqueness checks
Seller	GSTIN, contact email	Seller lookup and uniqueness checks
Category	ParentCategoryID, CategoryName	Hierarchy navigation and category search
Product	SellerID, CategoryID, Name	Catalog filtering and search
ProductImage	ProductID	Fast image retrieval
ProductEmbeddings	ProductID UNIQUE	Fast product-to-embedding lookup
PriceLedger	ProductID + Timestamp	Current/latest and history queries
Orders	CustomerID + OrderDate; ShippingStatus	Order history and status filtering
OrderDetails	ProductID; OrderID composite key	Product sales queries and line-item access

25. Database Views

- vw_product_catalog
- vw_current_product_price
- vw_product_price_history
- vw_order_summary
- vw_order_details
- vw_customer_order_summary
- vw_category_product_count
- vw_low_stock_products
- vw_seller_product_catalog
- vw_product_embedding_status

26. Stored Functions

```
fn_get_current_product_price()
fn_calculate_order_total()
fn_customer_order_count()
fn_customer_total_spent()
fn_product_stock_status()
```

27. Stored Procedures

```
sp_create_customer()
sp_add_customer_address()
sp_create_seller()
sp_create_category()
sp_create_product()
sp_add_product_image()
sp_add_product_embedding()
sp_add_price_ledger_entry()
sp_update_product_stock()
sp_place_order()
sp_update_shipping_status()
sp_cancel_order()
```

28. Database Triggers

- PriceLedger BEFORE UPDATE trigger to reject historical modification
- PriceLedger BEFORE DELETE trigger to enforce append-only history
- OrderDetails validation triggers

- Product stock validation triggers
- Timestamp maintenance triggers where appropriate

29. Backend Security & Authorization

- Spring Security integration
- Customer, Seller, and Admin roles
- Customers can access only their own profile, addresses, and orders
- Sellers can manage only their own products and inventory
- Marketplace/admin operations are restricted to authorized roles
- Secrets and passwords are provided through environment variables rather than source code
- Sensitive payment credentials are never stored

30. Validation & Exception Handling

Validation

@NotNull
 @NotBlank
 @Email
 @Positive
 @PositiveOrZero
 @Size
 @Pattern

Custom Exceptions

CustomerNotFoundException
 AddressNotFoundException
 SellerNotFoundException
 ProductNotFoundException
 CategoryNotFoundException
 OrderNotFoundException
 PriceNotFoundException
 InsufficientStockException
 DuplicateEmailException
 DuplicateGSTINException
 InvalidPriceException
 InvalidOrderException
 InvalidCategoryException
 UnauthorizedOperationException
 InvalidShippingStatusException

Global Error Handler

@RestControllerAdvice -> consistent JSON error responses without stack traces

31. Performance & Scalability

- Pagination for large product/order/customer collections
- DTO projections for read-heavy endpoints
- JOIN FETCH or EntityGraph where needed to prevent N+1 query problems
- Database indexes driven by actual queries
- Batch inserts for large seed/test datasets
- Avoid loading 100,000+ records into application memory at once
- Transactional row locking only where required

32. Frontend Pages

Area	Pages
Customer	Home, Login, Register, Products, Product Details, Category, Search, Visual Search, Cart, Checkout, Order Confirmation, My Orders, Order Details, Order Tracking, Profile, Addresses
Seller	Seller Login, Seller Dashboard, Seller Profile, Seller Products, Add Product, Edit Product, Inventory, Low Stock, Price Management, Price History, Seller Orders, Seller Sales
Marketplace	Marketplace/Admin Dashboard, Customer Management, Seller Management, Category Management, Product Management, Order Monitoring, Inventory Monitoring, Price Monitoring, Reporting

33. Frontend Components

Navbar
 Footer
 SearchBar
 CategoryMenu
 ProductCard
 ProductGrid
 ProductDetails
 ProductImageGallery
 PriceDisplay
 StockIndicator
 FilterPanel
 Pagination
 AddressForm
 AddressCard
 Cart
 CartItem
 CartSummary
 CheckoutForm
 OrderCard
 OrderDetails
 OrderStatusTracker
 SellerProductForm
 InventoryTable
 PriceHistoryTable
 VisualSearchUploader
 SearchResults
 DashboardCards

34. Frontend API Service Layer

customerApi
 addressApi
 sellerApi
 categoryApi
 productApi
 productImageApi
 inventoryApi
 priceApi
 embeddingApi

searchApi
cartApi
checkoutApi
orderApi
paymentApi
dashboardApi

35. Complete Customer Journey

Register -> Login -> Browse Categories -> Search Products
-> Normal Search OR Visual Search -> Product Details -> Select Quantity
-> Add to Cart -> Select Address -> Checkout -> Select Payment Method
-> Place Order -> Transaction -> Price Locked -> Stock Reduced
-> Order Created -> Confirmation -> Track Order

36. Complete Seller Journey

Register -> Seller Profile -> Create Product -> Select Category
-> Upload Images -> Create Embedding -> Add Initial Price -> Product Available
-> Manage Inventory -> Update Price -> Receive Orders -> Process Orders
-> Update Shipping Status -> View Sales

37. Complete Database Flow

```
Customer
|-- Address
|-- Orders
    |-- OrderDetails -- Product -- Seller
        |   |-- Category
        |   |-- ProductImage
        |   |-- ProductEmbeddings
        |   |-- PriceLedger
    |-- OrderPayment
```

38. Complete Feature & Service Inventory

Module	Frontend features	Backend services	Database scope
Customer	Registration, profile, contacts, multiple addresses, orders	CustomerService, AddressService, CustomerDashboardService	Customer, Address, CustomerEmail, CustomerPhone
Seller	Profile, products, inventory, prices, orders, sales	SellerService, SellerProductService, SellerOrderService, SellerDashboardService	Seller, Product, PriceLedger
Category	Hierarchy, subcategories, product navigation	CategoryService, CategoryHierarchyService	Category
Product	Catalog, details, images, stock, seller/category	ProductService, ProductCatalogService, ProductImageService	Product, ProductImage
Search	Keyword and filter search	ProductSearchService, CatalogSearchService	Product + indexes
Visual Search	Image upload, embeddings, similarity search	VisualSearchService, EmbeddingService, SimilaritySearchService	ProductEmbeddings
Pricing	Current price, history, price changes	PriceService, PriceLedgerService,	PriceLedger

Module	Frontend features	Backend services	Database scope
		CurrentPriceService	
Cart	Add/remove/update items (extension)	CartService, CartItemService	Application-level extension
Checkout	Address, item validation, payment, order creation	CheckoutService, OrderService, InventoryService	Orders, OrderDetails, OrderPayment
Orders	Create, history, details, status, cancel	OrderService, OrderDetailsService, OrderStatusService	Orders, OrderDetails
Payment	Payment method information	PaymentService, OrderPaymentService	OrderPayment
Reporting	Dashboards and marketplace analytics	ReportingService, AnalyticsService, CustomerDashboardService, SellerDashboardService	Views + queries
Database logic	Constraints, indexes, views, functions, procedures, triggers	DB layer	MySQL schema

39. Recommended Project Build Order

1. MySQL schema and constraints
2. JPA entities and repositories
3. Customer + Address
4. Seller + Category
5. Product + ProductImage
6. PriceLedger
7. ProductEmbeddings
8. Order + OrderDetails + Payment
9. Transactional checkout and inventory locking
10. REST controllers and DTOs
11. Security and authorization
12. Frontend catalog/search
13. Cart and checkout UI
14. Customer and seller dashboards
15. Visual search integration
16. Reporting, testing, and performance validation

40. Core Project Technical Highlights

- Normalized relational design
- Customer and multi-address management
- Seller-product management
- Hierarchical categories
- Product catalog and image management
- Inventory management
- AI product embeddings and visual similarity search
- Dynamic pricing with append-only PriceLedger
- Historical price preservation
- Transaction-time price locking through OrderDetails.ExactLedgerPrice
- Order/Product many-to-many resolution via OrderDetails
- Concurrency-safe inventory updates
- Order and shipping management
- Customer and seller dashboards
- Marketplace reporting
- MySQL constraints, indexes, views, functions, procedures, and triggers
- Spring Boot REST services and layered architecture
- Frontend REST integration

41. Source Basis & Scope Notes

This document consolidates the project features and services discussed for the OmniCart Marketplace. The documented core scope is based on the submitted Database Requirements Specification and Entity-Relationship Diagram. Where a feature is marked as an application-level extension (for example, a persistent cart or an explicit admin UI), it is included as an implementation option around the documented database model rather than as an entity explicitly required by the supplied ERD.

Source document	What it provides
Database Requirements Specification	Client/domain description and required customer, address, category, product, embedding, pricing, order, and price-locking behavior
Entity-Relationship Diagram	Core entities, keys, relationship cardinalities, and the OrderDetails / embedding / price-ledger relationships